Mira – Reflection and Next Steps

Q4 · Applied Agentic AI for PMs/TPMs capstone · Ramanpreet Singh Dandona · 12 November 2026

1. Trace findings

Mira passed 12 of 12 baseline tests with no hallucinations, after one fix found through the traces (runs on 11 and 12 Nov, traced in Langfuse).

- **T6 fix (evaluate → fix → re-evaluate):** "Generate a weekly status report for: 'Things are going fine.'" returned Out of Scope. The trace showed a Router call with no agent after it: the Router's vague-input rule had sent the request to "other". I added a rule to classify by the deliverable asked for, not the level of detail, and an example. On rerun Mira replied "NO TASK DATA AVAILABLE" and asked for a task board; T2, T4 and T9 still asked clarifying questions. Cost of the fix: +151 Router input tokens (535 → 686), about $0.00002 per request.
- **Code beats prompting for facts:** counts, overdue days and milestones are computed in a code node, so T10 matched the board exactly (5 Done, 3 In Progress, 1 Blocked, 16 To Do, 25 total).
- **Vague inputs are cheaper:** T2, T4 and T9 averaged about 1,200 tokens and $0.00022, against about 2,550 tokens and $0.00055 for detailed requests. A short clarifying question costs less than a long answer.
- **Per-agent view:** average $0.00044 and 4.7 s per request (Router plus agent). The Risk Assessor was the most expensive agent ($0.00040 per call; T3 was the costliest test at $0.00075 and 7.9 s); the Router was the cheapest ($0.00010, about 1.2 s).
- **Quality gaps that still passed:** T8 omitted overdue T006 and T008; T10 listed 2 completed tasks against a count of 5; T5 and T10 gave different health ratings (Red vs Amber).

2. Improvement plan

The next fixes, in priority order, each judged by rerunning the baseline tests before and after:

1. **Close the backlog:** Milestone Tracker always lists the overdue section and every task due in 7 days (T8); Status Reporter lists completed tasks from the whole board and applies one health rule (T5, T10).
2. **Golden test set on every change:** run T1–T12 automatically whenever a prompt or model changes, scored with Langfuse evaluations. The T6 fix showed why: one rule change can affect other vague-input tests.
3. **LLM-as-judge for groundedness:** check every task ID, date and milestone in an answer against the project data, so the hallucination rate becomes a tracked number.
4. **Inputs and cost:** accept flat files in CSV or Excel, with Jira or Trello as an optional connection; trim each agent's context to the fields it needs.

3. Evaluation and fine-tuning

Evaluation decides whether fine-tuning is needed, and supplies its training data: evaluate, find the failure, fix the prompt, re-evaluate, and fine-tune only what prompts can't fix.

- **Where it fits:** the best candidate is the agent with the most errors left after prompt fixes. In my run that was the Status Reporter (inconsistent completed lists and health ratings), with the Milestone Tracker next (T8 omissions).
- **Training data from production:** every PM correction and every failed test becomes a corrected example. I would build 10–15 input and output pairs, including 5 vague inputs where the correct answer is "insufficient data".
- **Measure it the same way:** run T1–T12 and the extended tests on the base and fine-tuned models, and compare pass rate, hallucinations, tokens and cost in Langfuse. Keep the fine-tuned model only if it is better on all of them.
- **Status:** fine-tuning was not attempted in this capstone (optional); prompt fixes resolved the only failure, T6.

4. Privacy considerations

Task boards and risk logs hold client names, people and commercial details, so Mira sends the minimum data to the LLM and keeps a human between Mira and the client.

- **Approval first:** legal and privacy approve sending client project data to the LLM provider before the pilot, under enterprise terms with no training on our data.
- **Data minimisation:** each agent receives only the fields it needs; the code layer already scopes context per agent.
- **Traces hold data too:** Langfuse stores inputs and outputs, so set retention limits, restrict access, and mask names in traces where possible.
- **Access control:** a PM can only query projects they are assigned to, enforced through role-based access on project files (and on the Jira or Trello token where that option is used).
- **No autonomous sending:** stakeholder emails stay drafts until the PM approves them; client updates leave out internal owner names.

5. Rollout for 10 PMs/TPMs

Roll out in stages, with a gate before each one, as set out in the program charter:

1. **Pilot (weeks 3–4):** 2 PMs on the ABCDE project, with every output reviewed. Continue only with zero fabrications, ≤ 3 edits per document and the time targets met.
2. **Harden (weeks 5–6):** fix pilot findings, add Excel input and an optional read-only Jira or Trello connection, add role-based access, and publish a one-page user guide.
3. **Rollout (weeks 7–8):** onboard 10 PMs/TPMs with a 30-minute training session and one champion PM per team, and put a feedback link in every reply.
4. **Measure (weeks 9–12):** track weekly usage (target 8 of 10), accuracy, time saved and cost per user in Langfuse; the CTO decides on wider rollout at week 12.

What I learned: the biggest gains came from deciding what *not* to give the LLM. Counting, dates and workload belong in code, and "not in the data" must be a valid answer. The model is the easy part; trust comes from the guardrails, the tests and the PM staying in the loop.